Design Doc: Swim Meet Optimization aka Nerd Shit

1. Executive Summary

The objective of this project is to architect and develop an automated, data driven system for optimizing competitive swim meet lineups (aka crushing Benny and Eddie). By treating lineup generation as a complex combinatorial optimization and sequential decision making problem, this system aims to maximize total team points while strictly adhering to city league constraints. This document outlines two parallel technological approaches: a stochastic Reinforcement Learning (RL) model utilizing **Gymnasium**, and a deterministic mathematical solver utilizing **Google OR-Tools**.

2. Problem Definition & Constraints

Competitive swimming requires coaches to assign a finite roster of athletes to a predefined schedule of events. The environment is heavily constrained and highly interactive.

- **Event Constraints:** Athletes are strictly limited to a maximum number of events (e.g., 2 individual events and 2 relays, or up to 4 total events per meet).
- **Fatigue Dynamics:** An athlete's performance degrades if assigned to back-to-back events (e.g., 200 IM followed immediately by the 50 Freestyle).
- **Scoring Mechanics:** Points are awarded based on relative finish position against an opposing team, not absolute time, making opponent strategy a critical variable.

3. Proposed Architecture & Technology Stack

To adequately address both the deterministic constraints and the stochastic elements of human performance, we propose a hybrid technology stack.

Component	Framework	Rationale & Application
RL Environment Interface	Gymnasium (Python)	Standardizes the state, action, and reward space. Essential for modeling opponent stochasticity and cumulative fatigue over the meet timeline.
RL Algorithms	Stable Baselines3 (PPO / MaskablePPO)	Proximal Policy Optimization is highly effective for discrete action spaces. MaskablePPO is strictly required to enforce legal lineup combinations.
Deterministic Optimization	Google OR-Tools (CP-SAT Solver)	Used as a baseline and fallback. Guarantees a mathematically optimal lineup when fatigue and opponent randomness are removed from the equation.
Data Pipeline & Processing	Pandas, NumPy, SciPy	Data cleaning, normalization of historical times, and generating probability distributions for opponent times.

4. Gymnasium Environment Specification (Markov

Decision Process)

The core of the Reinforcement Learning approach requires defining the meet as a Sequential Decision Process.

4.1 State Space (Observation)

The state vector must capture all necessary information for the agent to make a decision at any given step (event).

- **Roster Matrix:** An Nx3 matrix where N is the number of athletes. Columns represent: [Current Fatigue Level, Events Remaining, Lifetime Best Score].
- **Event Context:** One-hot encoded vector representing the current event being assigned (e.g., [1, 0, 0...] for 200 Medley Relay).
- **Opponent Projection:** Probabilistic finishing times for the opposing team in the current event based on historical scouting data.

4.2 Action Space & Action Masking

The action is the assignment of a specific swimmer (or set of swimmers for a relay) to the current event. Because an unconstrained model will invariably assign the fastest swimmer to every event, **Action Masking** is a critical architectural requirement. The environment will return an action_mask alongside the observation, setting the probability of illegal moves to zero.

```
# Example Gymnasium Env Structure for Swim Meet
import gymnasium as gym
from gymnasium import spaces
import numpy as np

class SwimMeetEnv(gym.Env):
    def __init__(self, roster_size, num_events):
        super().__init__()
        # Discrete action: Pick a swimmer from the roster (0 to N-1)
        self.action_space = spaces.Discrete(roster_size)

        # Observation space: Roster fatigue, event index, remaining eligibilities
        self.observation_space = spaces.Dict({
            "roster_status": spaces.Box(low=0, high=4, shape=(roster_size,), dtype=np.int8),
            "current_event": spaces.Discrete(num_events)
        })
```

```
def action_masks(self):
    # Returns a boolean array: True if swimmer is eligible, False otherwise
    return self.current_roster_eligibility > 0
```

4.3 Reward Function

The reward signal drives the agent's learning. The reward is formulated as:

- **Step Reward:** 0 during the assignment phase to prevent short-sighted optimization (greedy assignments in early events).
- **Terminal Reward:** Calculated at the end of the episode (the entire meet). It equals the simulated total meet points.
- **Penalty:** Extreme negative reward for any violation of physics or rules not caught by the action mask (fail-safe mechanism).

5. Core Technical Challenges & Mitigations

1. **The "Missing Data" / Sparsity Problem:**
    - *Challenge:* Swimmers rarely compete in every possible event, leaving gaps in the historical dataset (e.g., a sprinter with no 500 Freestyle time).
    - *Mitigation:* Implement a **Stroke Profiling Imputation Model**. Use linear regression against team benchmarks to extrapolate unknown times based on known primary events (e.g., predicting a 200 IM time based on a 100 Breaststroke and 100 Butterfly time).

2. **Modeling Fatigue Accurately:**
    - *Challenge:* Swimming a 500 Freestyle significantly impairs performance in subsequent events, but standard optimization solvers treat times statically.
    - *Mitigation:* Introduce a **Time Decay Function** into the Gymnasium state transition logic. If Athlete A swims Event X, their projected time for Event X+1 is multiplied by an empirically derived fatigue coefficient (e.g., 1.04 multiplier to their time).

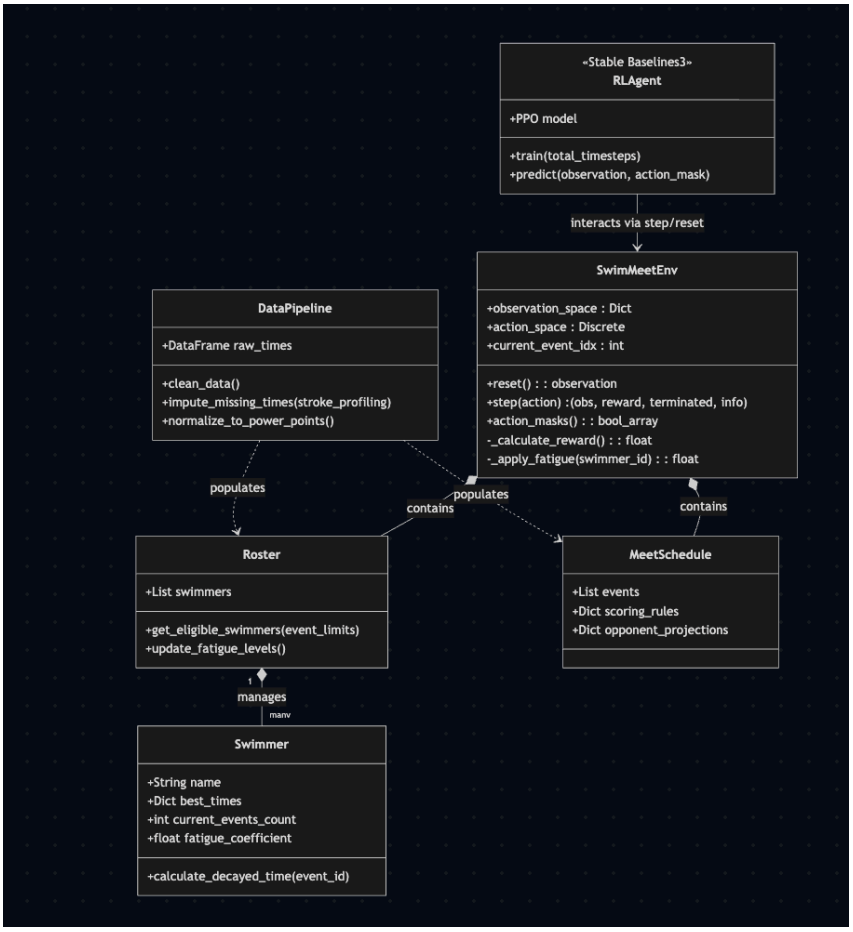
3. **Opponent Unpredictability:**
    - *Challenge:* We do not know the exact opponent lineup prior to the meet. Over-optimizing against a guessed opponent lineup leads to brittle strategies.

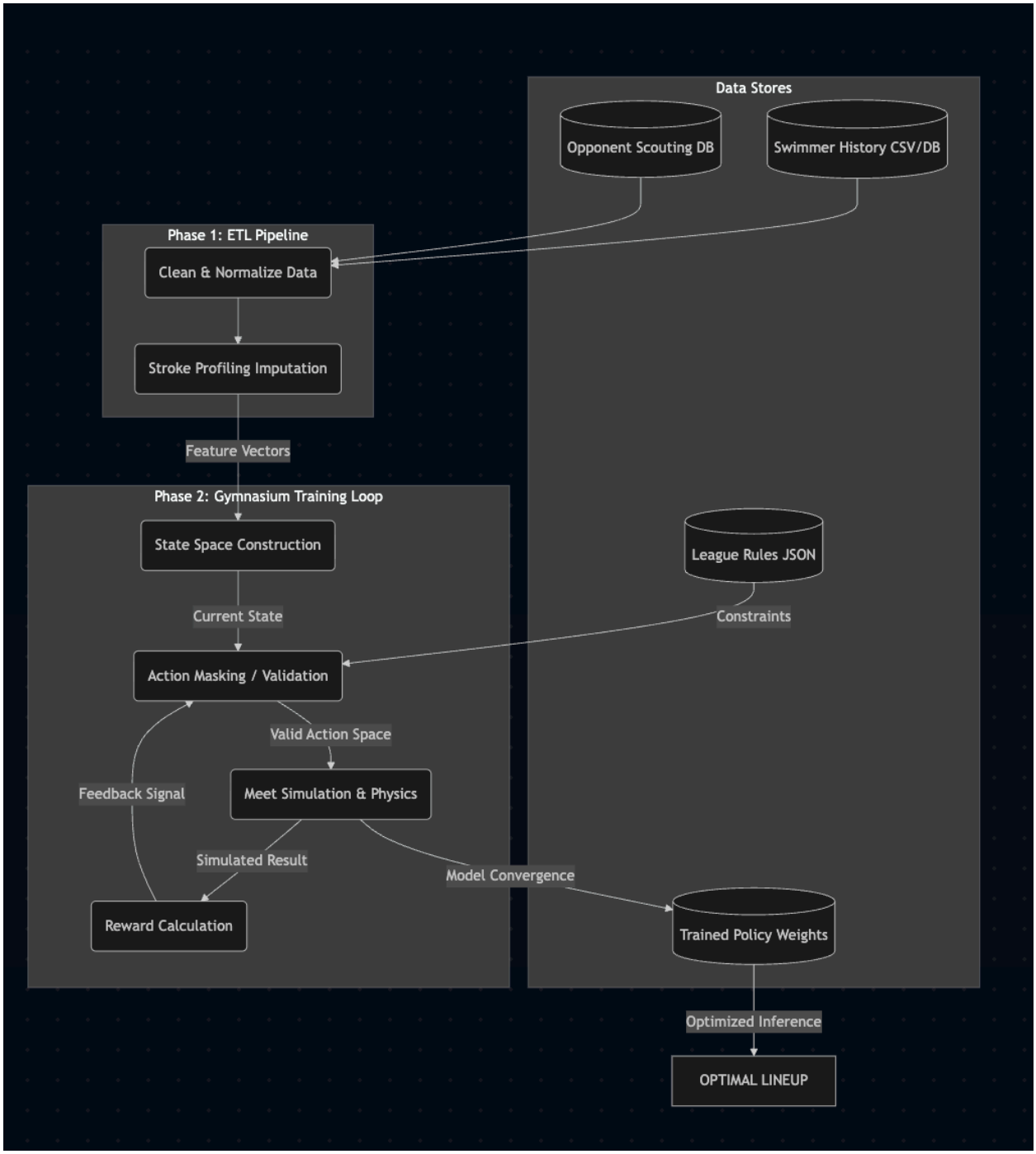
- *Mitigation:* Train the RL agent using **Domain Randomization**. The opponent's simulated lineup will be randomly sampled from a distribution of logical possible lineups across thousands of episodes. The agent will learn to build "robust" lineups that maximize average expected points across all probable opponent strategies.

6. Development Phases & Next Steps

- **Phase 1: Baseline Implementation.** Implement a deterministic baseline using Google OR-Tools. Establish the exact constraint logic (NFHS/NCAA rules) and validate against historical meets.
- **Phase 2: Gymnasium Prototype.** Build the custom Gym environment. Implement action masking to ensure 100% legal lineups. Run random actions to ensure environment stability.
- **Phase 3: RL Training & Tuning.** Integrate Stable Baselines3 MaskablePPO. Tune hyperparameters (learning rate, discount factor). Validate RL lineups against the OR-Tools baseline.
- **Phase 4: Simulation UI.** Build a simple frontend interface to input the upcoming opponent's roster and output the model's recommended lineup.

7. Diagrams




Data Stores
Opponent Scouting DB
Swimmer History CSV/DB
Phase 1: ETL Pipeline
Clean & Normalize Data
Stroke Profiling Imputation
Feature Vectors
Phase 2: Gymnasium Training Loop
State Space Construction
League Rules JSON
Current State
Constraints
Action Masking / Validation
Valid Action Space
Feedback Signal
Meet Simulation & Physics
Simulated Result
Model Convergence
Reward Calculation
Trained Policy Weights
Optimized Inference
OPTIMAL LINEUP